Reproducing a DDPG-Based Stock Trading Strategy

George Kontorousis
Department of Computer Science
McGill University
Montreal, Canada
george.kontorousis@mail.mcgill.ca

Yassine Meliani
Department of Computer Science
McGill University
Montreal, Canada
yassine.meliani@mail.mcgill.ca

Abstract

Stock trading is a challenging sequential decision-making problem because market dynamics are noisy and non-stationary. In this project, we reproduce the DDPG-based stock-trading formulation of Liu et al. [4], where an agent observes cash, stock prices, and portfolio holdings, then learns buy, sell, and hold decisions to maximize portfolio value. Using the original Dow Jones data split, we compare DDPG and TD3 against DJIA and a min-variance portfolio allocation strategy as baselines. Both reinforcement learning agents outperform the min-variance baseline, while TD3 achieves the strongest final portfolio value and Sharpe ratio. We also run a COVID-19 stress test (train 2010–2019, trade 2020), where RL agents do not consistently outperform DJIA and min-variance, indicating sensitivity to volatile regime shifts.

1 Background and Motivation

Recent work has modeled stock trading as a Markov decision process, allowing reinforcement learning methods to optimize sequential trading decisions directly rather than relying on separate return forecasting and portfolio optimization stages [4]. In the target paper, Liu et al. formulate the trading state using current stock prices, portfolio holdings, and remaining cash balance, and train a Deep Deterministic Policy Gradient (DDPG) agent to maximize changes in portfolio value over time [4]. Their results show that this approach outperforms both the Dow Jones Industrial Average (DJIA) and a classical min-variance portfolio baseline in terms of cumulative return and Sharpe ratio [4].

This project reproduces that stock-trading reinforcement learning setup using the original Dow Jones daily-price data and the same train/validation/test split. After preprocessing, the dataset used in our environment contains 28 tradable stocks. The goal is to evaluate whether the main performance trend reported by Liu et al. can be recovered with a modern implementation pipeline, while also comparing DDPG against Twin Delayed Deep Deterministic Policy Gradient (TD3), a more stable actor-critic variant designed to reduce common sources of instability in deterministic policy-gradient methods [2].

Rather than proposing a new trading algorithm, this work focuses on practical reproducibility and comparative evaluation. Financial reinforcement learning results are often sensitive to implementation details, hyperparameters, random seeds, and market dynamics, so we evaluate both learned agents against the same DJIA and min-variance baselines used in the original paper. Performance is measured using final portfolio value, annualized return, annualized volatility, and Sharpe ratio.

2 MDP Formulation and State Space

Following [4], we model stock trading as a finite-horizon Markov decision process. At each trading day, the agent observes the current portfolio, chooses buy/sell/hold actions for each stock, and receives

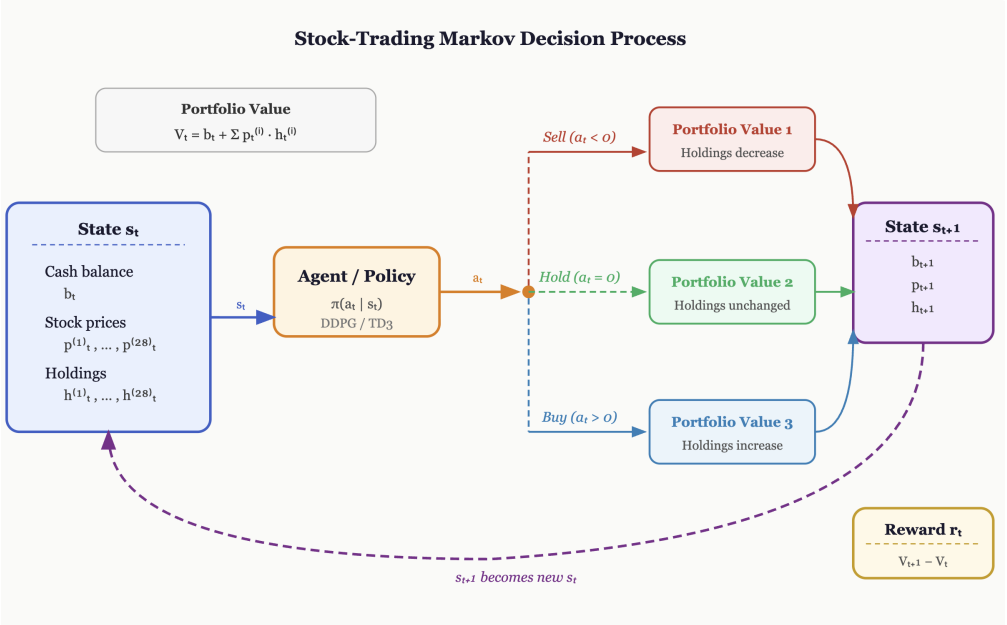

Figure 1: Stock-trading MDP used in our reproduction. The agent observes cash, current prices, and current holdings, then selects bounded buy/sell actions. The environment executes trades, advances to the next trading day, and returns the change in portfolio value as the reward.

a reward equal to the change in total portfolio value. Since the environment uses a fixed historical price series, the transition is deterministic once the dataset split and action are fixed.

The state contains cash, adjusted close prices, and holdings. Although the original paper refers to the Dow Jones 30 setting, our filtered dataset contains 28 tradable stocks, giving an observation vector of length 57:

$$s_t = \left[b_t, p_t^{(1)}, \ldots, p_t^{(28)}, h_t^{(1)}, \ldots, h_t^{(28)}\right],$$

where b_t is cash, $p_t^{(i)}$ is the adjusted close price of stock i, and $h_t^{(i)}$ is the number of shares held. Each episode starts with \$10,000 in cash and zero holdings.

The action is a 28-dimensional trade vector,

$$a_t = \left[a_t^{(1)}, \ldots, a_t^{(28)}\right],$$

with one action per stock. Actions are clipped, rounded, and executed as integers in

$$a_t^{(i)} \in \{-5, -4, \ldots, 0, \ldots, 4, 5\}.$$

Negative values sell shares, positive values buy shares, and zero holds the position. The environment is long-only: holdings cannot become negative, and buy orders are limited by available cash. We do not model transaction costs, slippage, taxes, or borrowing.

The portfolio value is

$$V_t = b_t + \sum_{i=1}^{28} p_t^{(i)} h_t^{(i)}.$$

After the agent acts, the environment executes trades at the current day's prices, advances to the next trading day, and computes the reward as

$$r_t = V_{t+1} - V_t.$$

Thus, the agent is directly rewarded for increasing mark-to-market portfolio value. Each episode is one pass through a data split: training (2009–2014), validation (2015), or testing (2016–2018), which is also referred to as 'trading'. As in the original paper, policy updates continue during trading, so this is an online evaluation rather than a fixed-policy test. The episode ends at the final trading day of the given split.

3 DDPG and TD3

We evaluate two off-policy actor-critic methods for continuous control: DDPG [3] and TD3 [2]. In both methods, an actor maps the state to a continuous action and a critic estimates state-action value; replay buffers and target networks are used to stabilize training. This makes both algorithms suitable for our trading environment, where the policy outputs one value per stock before the environment converts actions into bounded integer buy/sell/hold decisions.

TD3 extends DDPG to improve stability by using twin critics (with a clipped double-Q target), delayed actor updates, and target policy smoothing. We include DDPG as the primary reproduced baseline from Liu et al. [4], and TD3 as a stronger variant designed to reduce overestimation and noisy policy updates in this setting. Since DDPG and TD3 are well known algorithms, our discussion here is an overview of them; full algorithm outlines are provided in Appendix D.

4 Implementation

We used the Stable-Baselines3 implementations of DDPG and TD3 [6], running both algorithms through the same training and evaluation pipeline. For both models, we reconstructed the original stock-trading task as a custom Gymnasium environment [7], using daily Yahoo Finance market data fetched with yfinance [1] for the Dow Jones constituent stocks over the selected training date range.

We performed Bayesian hyperparameter optimization with Optuna [5], using 10 trials per algorithm. After selecting hyperparameters, each model was trained for 30 episodes and evaluated across three random seeds; results are reported as mean and standard deviation across seeds. Additional details about the hyperparameter tuning are provided in Appendix C.

5 Results and Discussion

Table 1 summarizes the final test-period performance of DDPG, TD3, the min-variance baseline, and the DJIA. Each reinforcement learning result is reported as the mean across three random seeds, with standard deviations shown in parentheses. Both RL agents outperformed the min-variance portfolio in final portfolio value and annualized return. DDPG slightly outperformed DJIA in annualized return while matching DJIA's Sharpe ratio. TD3 produced the strongest overall result, achieving the highest final portfolio value, annualized return, and Sharpe ratio; this higher Sharpe indicates better risk-adjusted performance. Representative portfolio-value curves from a specific seed are included in Appendix A; these show the same qualitative trend, with both learned agents tracking above the min-variance portfolio strategy baseline and TD3 ending above DDPG.

Table 1: Trading performance on the 2016–2018 test period. RL results are reported as mean across seeds, with standard deviation in parentheses.

Metric	DDPG	TD3	Min-Variance	DJIA
Initial Portfolio Value	\$10,000	\$10,000	\$10,000	\$10,000
Final Portfolio Value	\$16,141.47 (630.95)	\$16,918.53 (1002.53)	\$13,194.40	\$15,594.84
Annualized Return	19.25% (1.73)	21.31% (2.62)	10.74%	17.76%
Annualized Std.	12.78% (0.99)	12.67% (0.88)	9.37%	11.74%
Sharpe Ratio	1.51 (0.15)	1.68 (0.20)	1.15	1.51

The main split partially reproduces Liu et al. [4]: both RL agents outperform the min-variance baseline, and TD3 is strongest overall. However, our DDPG result (\$16,141.47 final value) remains below the original reported value (\$19,791), likely because our training and hyperparameter search budget is smaller and the original tuning procedure is not fully specified.

TD3 improves over DDPG in mean final portfolio value (\$16,918.53 vs. \$16,141.47) and mean Sharpe ratio (1.68 vs. 1.51), consistent with its stabilizing design (twin critics, delayed policy updates, and target smoothing) [2]. However, given the reported standard deviations (e.g., \$1002.53 vs. \$630.95 in final value), this advantage may be moderate rather than large. In this environment, where continuous actions are rounded into integer trades and rewards depend on noisy daily portfolio changes, these

stabilizing changes likely helped TD3 produce better risk-adjusted returns on average. More broadly, the results suggest that future work should focus not only on more complex models, but also on more realistic and robust evaluation, including longer training budgets, broader hyperparameter searches, transaction costs, slippage, and evaluation across additional market periods.

We also ran an additional stress test using the best hyperparameters from the main reproduction, training for 500 episodes on 2010–2019 and trading on the volatile 2020 COVID period (same Dow Jones stock universe; setup details in Appendix B). In this regime, RL did not consistently outperform classical baselines: TD3 underperformed both DJIA and min-variance (final value \$9,785.69, Sharpe -0.04), while DDPG had only a small mean final-value edge (\$10,590.07 vs. DJIA \$10,533.71 and min-variance \$10,476.71) but high seed variance and weaker risk-adjusted performance than min-variance. This highlights sensitivity to regime shifts and questions the robustness of the original effectiveness claim under highly volatile markets.

Table 2: COVID-19 stress-test performance on the 2020 trading period. RL results are mean across seeds, with standard deviation in parentheses.

Metric	DDPG	TD3	Min-Variance	DJIA
Initial Portfolio Value	\$10,000	\$10,000	\$10,000	\$10,000
Final Portfolio Value	\$10,590.07 (1261.53)	\$9,785.69 (305.36)	\$10,476.71	\$10,533.71
Annualized Return	5.93% (12.67)	-2.15% (3.07)	4.79%	5.36%
Annualized Std.	40.87% (6.30)	43.86% (6.03)	23.87%	36.88%
Sharpe Ratio	0.18 (0.32)	-0.04 (0.07)	0.20	0.15

6 Conclusion

We reproduced the stock-trading reinforcement learning setup of [4] using the same MDP formulation and historical Dow Jones data split. Our DDPG results followed the main trend of the original paper by outperforming the min-variance baseline, though they were weaker than the reported original results, likely due to a smaller training and hyperparameter-tuning budget and uncertainty around the original tuning procedure.

TD3 achieved the strongest performance in our experiments, outperforming DDPG in final portfolio value, annualized return, and Sharpe ratio. However, in an additional COVID-19 stress test on the volatile 2020 market, RL agents did not consistently outperform DJIA and min-variance baselines. Overall, these results suggest that while DDPG can learn useful trading behavior in this environment, TD3 provides a more stable and effective alternative for this reproduced stock-trading task, but robustness remains sensitive to market regime shifts.

Contributions and Acknowledgements

Both authors contributed equally to the implementation, experiments, analysis, and writing. We thank the authors of the original stock-trading DDPG repository and the developers of Stable-Baselines3, Gymnasium, yfinance and Optuna for the tools used in this project.

References

[1] Ran Aroussi. yfinance. `https://github.com/ranaroussi/yfinance`, 2024.

[2] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods, 2018. URL `https://arxiv.org/abs/1802.09477`.

[3] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning, 2015. URL `https://arxiv.org/abs/1509.02971`.

[4] Xiao-Yang Liu, Zhuoran Xiong, Shan Zhong, Hongyang Yang, and Anwar Walid. Practical deep reinforcement learning approach for stock trading, 2018. URL `https://arxiv.org/abs/1811.07522`.

[5] Yoshihiko Ozaki, Shuhei Watanabe, and Toshihiko Yanase. OptunaHub: A platform for black-box optimization. *arXiv preprint arXiv:2510.02798*, 2025.

[6] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL `http://jmlr.org/papers/v22/20-1364.html`.

[7] Mark Towers, Ariel Kwiatkowski, Jordan K. Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulao, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium, 2023. URL `https://zenodo.org/record/8127026`.

A Portfolio Value Plots

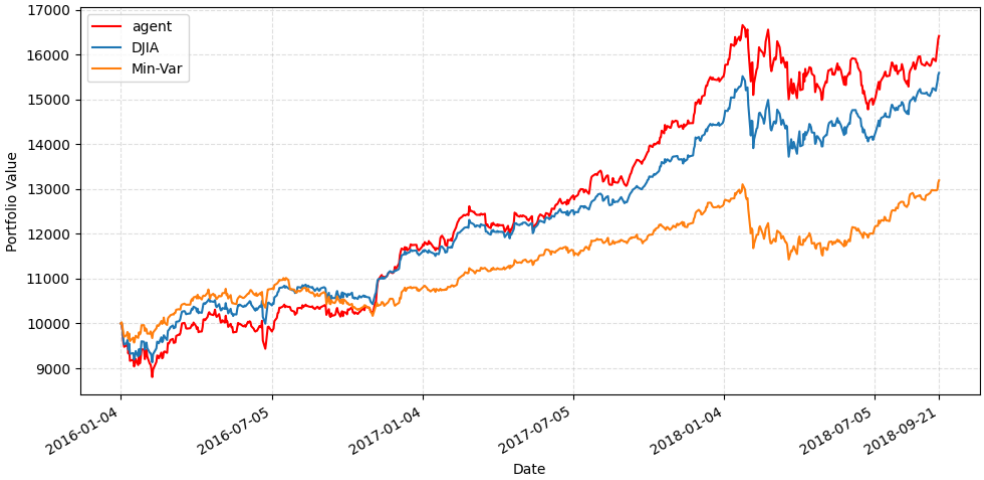

Figure 2: DDPG portfolio value over the trading period compared against DJIA and the min-variance baseline. (seed 42)

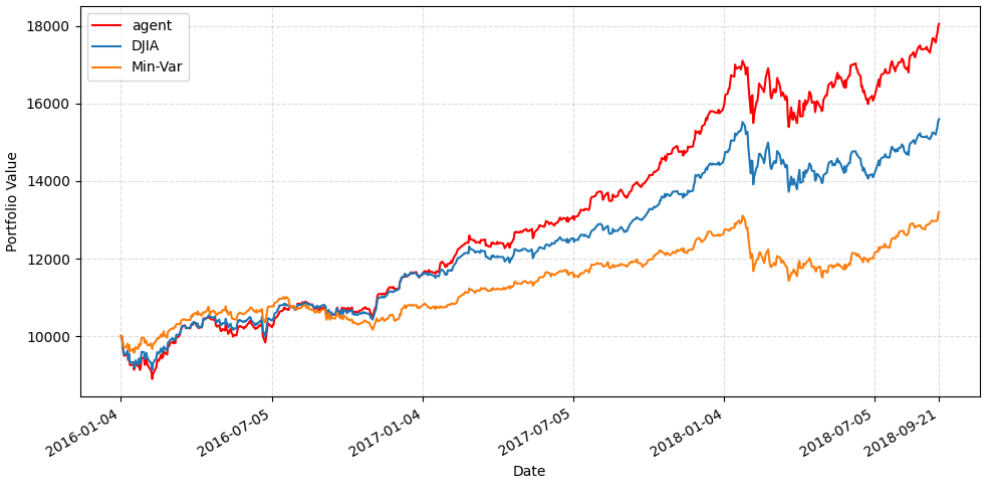

Figure 3: TD3 portfolio value over the trading period compared against DJIA and the min-variance baseline. (seed 42)

B COVID-19 Stress-Test Setup and Results

We implemented the stress test in a similar pipeline as the main reproduction experiment. For each algorithm, we reused the best hyperparameters from the main reproduction (no re-tuning), then trained on 2010–2019 daily Dow Jones data and traded online on the 2020 COVID period, using the same Dow Jones universe, \$10,000 initial balance, and max 5 shares per trade (per ticker/stock). We evaluated three seeds (42, 43, 44), corresponding to 500 episodes (1,257,500 timesteps), and report mean and standard deviation across seeds for RL agents.

C Hyperparameter Optimization Details

We tuned both DDPG and TD3 with Optuna over the same search ranges used in the implementation: learning rate in $[10^{-4}, 10^{-3}]$, batch size in $\{64, 128, 256\}$, discount factor γ in $[0.95, 0.999]$, soft-

update rate τ in $[0.001, 0.01]$, and exploration-noise scale σ in $[0.05, 0.3]$. For trial selection, we used Optuna's TPE sampler (`TPESampler`), which is a Bayesian hyperparameter optimization method.

Best hyperparameters from the 30-episode/10-trial main reproduction runs were:

- **DDPG:** learning rate $= 3.6328 \times 10^{-4}$, batch size $= 256$, $\gamma = 0.9533$, $\tau = 0.00568$, noise $\sigma = 0.21499$.
- **TD3:** learning rate $= 2.3150 \times 10^{-4}$, batch size $= 64$, $\gamma = 0.96165$, $\tau = 0.00537$, noise $\sigma = 0.29054$.

D Algorithm Details

Algorithm 1 DDPG

Initialize actor network $\mu(s|\theta^\mu)$ and critic network $Q(s, a|\theta^Q)$
Initialize target networks μ' and Q' with the same weights
Initialize replay buffer $\mathcal{R}$
for each episode **do**
    Receive initial state s_1
    for each time step t **do**
        Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$
        Execute action a_t, observe reward r_t and next state s_{t+1}
        Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{R}$
        Sample a mini-batch of transitions from $\mathcal{R}$
        Compute target:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}))$$

        Update critic by minimizing:

$$L = \frac{1}{N} \sum_i \left(y_i - Q(s_i, a_i)\right)^2$$

        Update actor using the policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s_i)$$

        Soft-update target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}, \qquad \theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

    end for
end for

Algorithm 2 TD3

Initialize actor network $\mu(s|\theta^\mu)$
Initialize two critic networks $Q_1(s,a|\theta^{Q_1})$ and $Q_2(s,a|\theta^{Q_2})$
Initialize target networks μ', Q'_1, and Q'_2
Initialize replay buffer $\mathcal{R}$
for each episode **do**
    Receive initial state s_1
    for each time step t **do**
        Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$
        Execute action a_t, observe reward r_t and next state s_{t+1}
        Store transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{R}$
        Sample a mini-batch of transitions from $\mathcal{R}$
        Add clipped noise to the target action:

$$\tilde{a}_{i+1} = \mu'(s_{i+1}) + \epsilon, \qquad \epsilon \sim \mathrm{clip}(\mathcal{N}(0,\sigma), -c, c)$$

        Compute target using the smaller critic value:

$$y_i = r_i + \gamma \min_{j=1,2} Q'_j(s_{i+1}, \tilde{a}_{i+1})$$

        Update both critics by minimizing:

$$L_j = \frac{1}{N} \sum_i \left(y_i - Q_j(s_i, a_i)\right)^2, \qquad j \in \{1,2\}$$

        if policy update step **then**
            Update actor using Q_1:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q_1(s,a)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s_i)$$

            Soft-update target networks
        end if
    end for
end for